

Đồ thị và các thuật toán cơ bản (Graphs and Basic Graph Algorithms)

Giảng viên: Trần Đình Tân

Môn: Thuật toán ứng dụng

Sau bài học, người học có thể:

- Mô hình hóa được bài toán thực tế bằng đồ thị $G = (V, E)$.
- Phân biệt được đồ thị vô hướng, có hướng, có trọng số và không trọng số.
- Biểu diễn được đồ thị bằng danh sách kề và ma trận kề; chọn biểu diễn phù hợp.
- Trình bày, mô phỏng và cài đặt được BFS và DFS.
- Ứng dụng duyệt đồ thị để tìm đường đi, kiểm tra liên thông, thành phần liên thông và đồ thị hai phía.
- Phân tích được độ phức tạp theo $|V|$ và $|E|$.

Nội dung

- 1 Mô hình đồ thị
- 2 Biểu diễn đồ thị
- 3 BFS
- 4 DFS
- 5 Ứng dụng và thực hành
- 6 Tổng kết
- 7 Bài tập và tài liệu

Đồ thị là gì?

Định nghĩa

Trong bài này, một đồ thị đơn hữu hạn $G = (V, E)$ gồm tập đỉnh V và tập cạnh E . Đỉnh biểu diễn đối tượng; cạnh biểu diễn quan hệ giữa các đối tượng.

- Ví dụ đỉnh: thành phố, người dùng, trang web, môn học, trạng thái của trò chơi.
- Ví dụ cạnh: đường đi, quan hệ bạn bè, liên kết web, quan hệ tiên quyết, phép chuyển trạng thái.
- Khi mô hình hóa, cần trả lời ba câu hỏi: *đỉnh là gì, cạnh là gì, và bài toán cần tối ưu/kiểm tra điều gì.*

Bài toán

Cho bản đồ các thành phố và đường nối. Cần tìm xem có thể đi từ s đến t hay không.

- Đỉnh: các thành phố.
- Cạnh: các đường nối trực tiếp.
- Truy vấn: tồn tại đường đi từ s đến t ?

Mô hình đồ thị

$$V = \{A, B, C, D, E\}$$

$$E = \{AB, AC, BD, CE, DE\}.$$

- Nếu đường hai chiều: đồ thị vô hướng.
- Nếu đường một chiều: đồ thị có hướng.

Các loại đồ thị thường gặp

Loại đồ thị	Ý nghĩa cạnh	Ví dụ bài toán
Vô hướng	Quan hệ hai chiều	mạng bạn bè, đường hai chiều
Có hướng	Quan hệ một chiều	liên kết web, phụ thuộc công việc
Không trọng số	Mỗi cạnh có chi phí như nhau	số bước ít nhất
Có trọng số	Cạnh có chi phí/độ dài	đường đi ngắn nhất theo chi phí

Các khái niệm cơ bản

Các khái niệm này giúp phát biểu chính xác bài toán và điều kiện đúng của thuật toán.

- **Bậc** của một đỉnh trong đồ thị vô hướng là số cạnh kề với đỉnh đó.
- Với đồ thị có hướng, phân biệt **bậc vào** và **bậc ra**.
- **Đường đi** là dãy đỉnh $v_0, v_1, \dots, v_k$ sao cho mỗi cặp liên tiếp có cạnh nối; với đồ thị có hướng, cạnh phải đi theo chiều $v_i \rightarrow v_{i+1}$.
- **Chu trình** là đường đi khép kín, tức $v_0 = v_k$, với các cạnh hợp lệ; chu trình đơn thường không lặp đỉnh trung gian.
- **Liên thông** thường nói cho đồ thị vô hướng: mọi cặp đỉnh đều có đường đi nối nhau. Với đồ thị có hướng, cần phân biệt liên thông mạnh và liên thông yếu.

Từ bài toán thực tế sang bài toán đồ thị

- ❶ Xác định đối tượng nào cần được biểu diễn thành đỉnh.
- ❷ Xác định quan hệ nào cần được biểu diễn thành cạnh.
- ❸ Quyết định cạnh có hướng hay vô hướng, có trọng số hay không.
- ❹ Xác định đầu vào, đầu ra và tiêu chí đúng.
- ❺ Chọn thuật toán phù hợp với giả định của mô hình.

Ví dụ kiểm tra nhanh

Nếu mục tiêu là “ít số chặng nhất” trong mạng không trọng số, BFS là ứng viên tự nhiên. Nếu mục tiêu là “ít chi phí nhất” với chi phí khác nhau, BFS không còn đủ.

Danh sách kề

Ý tưởng

Với mỗi đỉnh u , lưu danh sách các đỉnh kề với u .

- Tiết kiệm bộ nhớ cho đồ thị thưa.
- Duyệt các cạnh đi ra từ một đỉnh rất nhanh.
- Phù hợp với BFS/DFS.
- Bộ nhớ: $O(|V| + |E|)$.

Ví dụ

A	:	B, C
B	:	A, D
C	:	A, E
D	:	B, E
E	:	C, D

Ý tưởng

Với đồ thị không trọng số, dùng ma trận M kích thước $|V| \times |V|$, trong đó $M[u][v] = 1$ nếu có cạnh từ u đến v . Với đồ thị có trọng số, $M[u][v]$ thường lưu trọng số hoặc một giá trị đặc biệt nếu không có cạnh.

- Kiểm tra cạnh (u, v) trong $O(1)$.
- Tốn $O(|V|^2)$ bộ nhớ, kể cả khi số cạnh rất ít.
- Phù hợp với đồ thị dày hoặc bài toán cần truy vấn cạnh rất nhiều lần.
- Với đồ thị vô hướng, ma trận kề đối xứng qua đường chéo chính.

So sánh hai cách biểu diễn

Tiêu chí	Danh sách kề	Ma trận kề
Bộ nhớ	$O(V + E)$	$O(V ^2)$
Kiểm tra cạnh (u, v)	thường $O(\deg(u))$	$O(1)$
Duyệt hàng xóm của u	$O(\deg(u))$	$O(V)$
Phù hợp	đồ thị thưa	đồ thị dày
BFS/DFS	thường dùng	ít hiệu quả hơn khi thưa

Kết luận thực hành

Trong hầu hết bài toán duyệt đồ thị kích thước lớn, danh sách kề là lựa chọn mặc định.

Trực giác

Một đồ thị đơn với $n = |V|$ có tối đa $n(n-1)/2$ cạnh nếu vô hướng và $n(n-1)$ cung nếu có hướng; cả hai đều là $O(n^2)$. Nếu $|E|$ nhỏ hơn nhiều so với $|V|^2$, ta gọi là đồ thị thưa.

- Mạng xã hội lớn thường thưa: mỗi người chỉ kết nối với một phần rất nhỏ toàn bộ mạng.
- Ma trận kề cho mạng lớn dễ gây lãng phí bộ nhớ.
- Nếu $|V| = 10^5$, ma trận $|V| \times |V|$ gần như không khả thi trong bộ nhớ thông thường.
- Phân tích theo $|V|$ và $|E|$ giúp thấy rõ lựa chọn biểu diễn có ảnh hưởng lớn đến độ phức tạp.

Ý tưởng BFS

Ý tưởng chính

BFS duyệt đồ thị theo từng lớp khoảng cách từ đỉnh nguồn s .

- Lớp 0: đỉnh nguồn s .
- Lớp 1: các đỉnh kề trực tiếp với s .
- Lớp 2: các đỉnh chưa thăm kề với lớp 1.
- Tiếp tục cho đến khi không còn đỉnh mới.

Tính chất quan trọng

Khi một đỉnh v được phát hiện lần đầu trong BFS, khoảng cách $dist[v]$ đã là số cạnh ít nhất từ s đến v trong đồ thị không trọng số.

BFS: cấu trúc dữ liệu

- **Queue**: lưu các đỉnh đang chờ mở rộng.
- **visited**: đánh dấu đỉnh đã được phát hiện để tránh duyệt lặp.
- **dist**: lưu khoảng cách theo số cạnh từ nguồn.
- **parent**: lưu đỉnh trước đó để truy vết đường đi.

Quy tắc cập nhật

Khi xét cạnh (u, v) , nếu v chưa thăm thì đặt

$$visited[v] = true, \quad dist[v] = dist[u] + 1, \quad parent[v] = u.$$

BFS: pseudocode

```
1: Khởi tạo  $visited[v] \leftarrow false$ ,  $dist[v] \leftarrow \infty$ ,  $parent[v] \leftarrow NIL$  với mọi  $v \in V$ 
2:  $visited[s] \leftarrow true$ ,  $dist[s] \leftarrow 0$ ,  $parent[s] \leftarrow NIL$ 
3: Đưa  $s$  vào hàng đợi  $Q$ 
4: while  $Q$  không rỗng do
5:   Lấy  $u$  khỏi đầu hàng đợi  $Q$ 
6:   for mỗi đỉnh  $v$  kề với  $u$  do
7:     if  $visited[v] = false$  then
8:        $visited[v] \leftarrow true$ 
9:        $dist[v] \leftarrow dist[u] + 1$ 
10:       $parent[v] \leftarrow u$ 
11:      Đưa  $v$  vào cuối hàng đợi  $Q$ 
12:    end if
13:  end for
14: end while
```

Ví dụ chạy BFS bằng tay

Xét danh sách kề:

$$A : \{B, C\}, \quad B : \{A, D\}, \quad C : \{A, E\}, \quad D : \{B, E\}, \quad E : \{C, D\}.$$

Chạy BFS từ A:

Bước	Lấy khỏi queue	Đỉnh mới phát hiện	Queue sau bước
0	–	A	[A]
1	A	B, C	[B, C]
2	B	D	[C, D]
3	C	E	[D, E]
4	D	–	[E]
5	E	–	[]

Truy vết đường đi bằng BFS

Ý tưởng

Mảng *parent* tạo thành cây BFS. Muốn tìm đường từ *s* đến *t*, đi ngược từ *t* qua *parent[t]* cho đến *s*.

- Nếu *visited[t] = false* sau BFS, không tồn tại đường đi từ *s* đến *t*.
- Nếu *visited[t] = true*, dãy truy vết ngược cho một đường đi ngắn nhất theo số cạnh.
- Cần đảo ngược dãy truy vết để in đường đi theo chiều từ *s* đến *t*.

Ví dụ

Nếu *parent[E] = C* và *parent[C] = A*, đường đi từ *A* đến *E* là $A \rightarrow C \rightarrow E$.

- **Tìm đường đi ngắn nhất theo số cạnh:** áp dụng cho đồ thị không trọng số.
- **Kiểm tra khả năng tới được:** từ một đỉnh nguồn có thể đến những đỉnh nào; với đồ thị có hướng phải tôn trọng chiều cạnh.
- **Tìm thành phần liên thông:** trong đồ thị vô hướng, chạy BFS từ các đỉnh chưa thăm.
- **Kiểm tra đồ thị hai phía:** tô màu xen kẽ theo lớp BFS và lặp lại cho mọi thành phần chưa tô màu.
- **Tìm số bước tối thiểu trong không gian trạng thái:** mỗi trạng thái là một đỉnh, mỗi phép biến đổi là một cạnh.

Độ phức tạp của BFS

Với danh sách kề

Mỗi đỉnh được đưa vào queue nhiều nhất một lần; mỗi cạnh vô hướng được xét tối đa hai lần, còn mỗi cung có hướng được xét tối đa một lần theo danh sách kề.

- Thời gian: $O(|V| + |E|)$.
- Bộ nhớ phụ: $O(|V|)$ cho *visited*, *dist*, *parent* và queue.
- Nếu dùng ma trận kề, việc duyệt hàng xóm tốn $O(|V|)$ cho mỗi đỉnh, tổng thời gian có thể thành $O(|V|^2)$.

Ý tưởng DFS

Ý tưởng chính

DFS đi sâu nhất có thể theo một nhánh, khi không thể đi tiếp thì quay lui.

- Có thể cài bằng đệ quy hoặc bằng ngăn xếp.
- DFS tạo ra cây/rừng DFS và thứ tự vào/ra của các đỉnh.
- Hữu ích trong phát hiện chu trình, thành phần liên thông, sắp xếp topo và nhiều thuật toán nâng cao.

Khác với BFS

DFS không đảm bảo tìm đường đi ngắn nhất theo số cạnh trong đồ thị không trọng số.

DFS: pseudocode đệ quy

```
1: Khởi tạo  $visited[v] \leftarrow false$  với mọi  $v \in V$ 
2:
3: Procedure  $DFS(u)$ 
4:    $visited[u] \leftarrow true$ 
5:   Xử lý đỉnh  $u$ 
6:   for mỗi đỉnh  $v$  kề với  $u$  do
7:     if  $visited[v] = false$  then
8:        $parent[v] \leftarrow u$ 
9:        $DFS(v)$ 
10:    end if
11:  end for
```

Chạy toàn đồ thị

Để duyệt cả đồ thị không liên thông, gọi $DFS(u)$ với mọi đỉnh u chưa thăm.

Ví dụ chạy DFS bằng tay

Với cùng đồ thị và thứ tự duyệt hàng xóm theo chữ cái:

$$A : \{B, C\}, \quad B : \{A, D\}, \quad C : \{A, E\}, \quad D : \{B, E\}, \quad E : \{C, D\}.$$

Chạy DFS từ A có thể tạo thứ tự:

$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C.$$

- Thứ tự DFS phụ thuộc vào thứ tự liệt kê hàng xóm.
- Cây DFS trong ví dụ có các cạnh cây: AB, BD, DE, EC .
- Cạnh còn lại có thể cho thấy sự tồn tại của chu trình trong đồ thị vô hướng.

Đồ thị vô hướng

Khi duyệt cạnh (u, v) , nếu v đã thăm và $v \neq \text{parent}[u]$, ta phát hiện một chu trình.

- Điều kiện $v \neq \text{parent}[u]$ là cần thiết vì cạnh quay về cha trong cây DFS không tạo chu trình mới.
- Với đồ thị có hướng, cần thêm trạng thái màu: trắng, xám, đen.
- Gặp cạnh từ đỉnh đang xét đến một đỉnh màu xám là dấu hiệu có chu trình có hướng.

Bài toán

Cho đồ thị có hướng không chu trình (DAG). Cần sắp xếp các đỉnh sao cho mọi cạnh $u \rightarrow v$ đều có u đứng trước v .

- Ví dụ: thứ tự học các môn có quan hệ tiên quyết.
- Với DAG, DFS có thể sinh thứ tự topo bằng cách đưa đỉnh vào danh sách sau khi đã duyệt xong các đỉnh kề.
- Đảo ngược danh sách hoàn tất sẽ thu được một thứ tự topo.
- Nếu đồ thị có chu trình, thứ tự topo không tồn tại.

Độ phức tạp của DFS

Với danh sách kề

DFS cũng duyệt mỗi đỉnh và mỗi cạnh một số lần hằng.

- Thời gian: $O(|V| + |E|)$.
- Bộ nhớ đánh dấu: $O(|V|)$.
- Nếu dùng đệ quy, độ sâu ngăn xếp lời gọi có thể là $O(|V|)$ trong trường hợp xấu.
- Với đồ thị rất sâu, có thể cần cài DFS bằng ngăn xếp tường minh để tránh tràn stack.

BFS và DFS: so sánh nhanh

Tiêu chí	BFS	DFS
Cấu trúc dữ liệu	Queue	Stack/đệ quy
Chiến lược	Theo lớp	Đi sâu rồi quay lui
Đường đi ngắn nhất không trọng số	Có	Không đảm bảo
Phát hiện chu trình	Có thể	Rất tự nhiên
Thành phần liên thông	Có	Có
Độ phức tạp	$O(V + E)$	$O(V + E)$

Khi nào dùng BFS, khi nào dùng DFS?

- Dùng **BFS** khi cần khoảng cách theo số cạnh, đường đi ít bước nhất, hoặc duyệt theo từng lớp.
- Dùng **DFS** khi cần quay lui, phát hiện chu trình, phân tích cấu trúc sâu, hoặc sắp xếp topo.
- Cả hai đều dùng được để kiểm tra liên thông và tìm thành phần liên thông trong đồ thị vô hướng; với đồ thị có hướng, kết quả chỉ là tập đỉnh tới được nếu chạy từ một nguồn.
- Trước khi chọn thuật toán, luôn kiểm tra: đồ thị có trọng số không, có hướng không, cần đường đi ngắn nhất hay chỉ cần khả năng tới được.

Định nghĩa

Trong đồ thị vô hướng, một thành phần liên thông là một tập đỉnh cực đại sao cho mọi cặp đỉnh trong tập đều có đường đi nối nhau.

- Với đồ thị vô hướng, có thể tìm bằng cách lặp BFS hoặc DFS.
- Mỗi lần bắt đầu từ một đỉnh chưa thăm, ta thu được một thành phần mới.
- Tổng thời gian vẫn là $O(|V| + |E|)$ vì mỗi đỉnh và mỗi cạnh chỉ được xét số lần hằng.

Thuật toán tìm thành phần liên thông

```
1:  $component\_id \leftarrow 0$ 
2: Đặt  $visited[v] \leftarrow false$  với mọi  $v \in V$ 
3: for mỗi đỉnh  $u \in V$  do
4:   if  $visited[u] = false$  then
5:      $component\_id \leftarrow component\_id + 1$ 
6:     Chạy BFS/DFS từ  $u$ 
7:     Gán tất cả đỉnh vừa thăm được vào thành phần  $component\_id$ 
8:   end if
9: end for
```

Ý nghĩa

Nếu số thành phần bằng 1, đồ thị vô hướng là liên thông. Nếu lớn hơn 1, đồ thị bị tách thành nhiều cụm.

Kiểm tra đồ thị hai phía bằng BFS

Định nghĩa

Đồ thị vô hướng hai phía là đồ thị có thể chia đỉnh thành hai tập L và R sao cho mọi cạnh đều nối một đỉnh ở L với một đỉnh ở R .

- Dùng BFS để tô màu hai màu xen kẽ theo lớp; nếu đồ thị không liên thông, bắt đầu BFS mới từ mỗi đỉnh chưa tô màu.
- Khi duyệt cạnh (u, v) :
 - nếu v chưa màu, gán màu khác màu u ;
 - nếu v đã có cùng màu với u , đồ thị không hai phía.
- Ứng dụng: mô hình quan hệ giữa hai loại đối tượng, ví dụ sinh viên–môn học hoặc người–sự kiện.

Bài toán khởi động trên lớp

Đề bài

Cho đồ thị vô hướng có

$$V = \{1, 2, 3, 4, 5, 6\}$$

và

$$E = \{12, 13, 24, 35, 45, 56\}.$$

Hãy chạy BFS từ đỉnh 1.

- 1 Ghi thứ tự lấy đỉnh ra khỏi queue.
- 2 Tính $dist[v]$ với mọi đỉnh v .
- 3 Dùng $parent$ để truy vết một đường đi ngắn nhất từ 1 đến 6.

Mô phỏng BFS: bảng trạng thái

Bước	Đỉnh đang xét	Queue trước khi xét	Cập nhật chính
1	1	[1]	phát hiện 2, 3
2	2	[2, 3]	phát hiện 4
3	3	[3, 4]	phát hiện 5
4	4	[4, 5]	không có đỉnh mới
5	5	[5]	phát hiện 6
6	6	[6]	không có đỉnh mới

Kết quả

Một đường đi ngắn nhất từ 1 đến 6 là $1 \rightarrow 3 \rightarrow 5 \rightarrow 6$ với độ dài 3 cạnh.

Cài đặt: các lỗi thường gặp

- Đánh dấu *visited* quá muộn: nếu chỉ đánh dấu khi lấy khỏi queue, một đỉnh có thể bị đưa vào queue nhiều lần.
- Quên khởi tạo *dist* bằng ∞ hoặc giá trị đặc biệt cho đỉnh chưa tới được.
- Nhầm đồ thị vô hướng và có hướng khi thêm cạnh vào danh sách kề.
- Với DFS đệ quy, quên điều kiện dừng hoặc không đánh dấu trước khi gọi đệ quy.
- Dùng BFS cho đồ thị có trọng số rồi kết luận sai về đường đi ngắn nhất theo chi phí.

Kiểm thử trường hợp biên

- Đồ thị rỗng hoặc chỉ có một đỉnh.
- Đồ thị không có cạnh.
- Đồ thị có nhiều thành phần liên thông.
- Đỉnh nguồn và đỉnh đích trùng nhau.
- Có đường đi nhưng không duy nhất.
- Đồ thị có hướng nhưng dữ liệu được nhập như đồ thị vô hướng, hoặc ngược lại.

Nguyên tắc

Không chỉ kiểm thử trường hợp “đẹp”. Các lỗi quan trọng thường xuất hiện ở dữ liệu nhỏ, rời rạc hoặc sát điều kiện biên.

- ❶ Vì sao BFS tìm được đường đi ngắn nhất theo số cạnh trong đồ thị không trọng số?
- ❷ Cho ví dụ DFS tìm được một đường đi nhưng không phải đường đi ngắn nhất.
- ❸ Khi nào ma trận kề tốt hơn danh sách kề?
- ❹ Nếu đồ thị có 10^6 đỉnh nhưng chỉ có $3 \cdot 10^6$ cạnh, nên chọn biểu diễn nào?
- ❺ Cần thay đổi gì nếu đồ thị là có hướng?

Chuẩn bị cho lab

- Cài cấu trúc danh sách kề cho đồ thị vô hướng và có hướng.
- Viết hàm BFS trả về *dist* và *parent*.
- Viết hàm khôi phục đường đi từ *s* đến *t* bằng mảng *parent*.
- Viết hàm DFS để đếm số thành phần liên thông của đồ thị vô hướng.
- So sánh thời gian chạy khi tăng $|V|$ và $|E|$.

Thông điệp chính

BFS và DFS là hai phép duyệt nền tảng của phần lớn thuật toán đồ thị.

- Mô hình đồ thị quyết định thuật toán nào được phép dùng.
- Danh sách kề thường phù hợp cho đồ thị thưa và các bài toán duyệt.
- BFS duyệt theo lớp và cho đường đi ngắn nhất theo số cạnh trong đồ thị không trọng số.
- DFS đi sâu rồi quay lui, phù hợp cho phân tích cấu trúc, chu trình và thứ tự topo.
- Luôn phân tích độ phức tạp theo $|V|$ và $|E|$.

- 1 Cho đồ thị $V = \{A, B, C, D, E, F\}$ và $E = \{AB, AC, BD, CE, DE, EF\}$. Chạy BFS từ A và lập bảng queue ở từng bước.
- 2 Với cùng đồ thị, chạy DFS từ A theo thứ tự chữ cái và ghi cây DFS thu được.
- 3 Chứng minh ngắn gọn vì sao BFS có thời gian $O(|V| + |E|)$ nếu dùng danh sách kề.
- 4 Tìm các thành phần liên thông của một đồ thị có $V = \{1, 2, 3, 4, 5, 6, 7\}$ và $E = \{12, 23, 45, 56\}$.

- 1 Cài đặt BFS và DFS bằng danh sách kề cho đồ thị vô hướng.
- 2 Viết hàm `shortestPath(s,t)` trả về đường đi ngắn nhất theo số cạnh hoặc thông báo không tồn tại.
- 3 Viết hàm kiểm tra đồ thị hai phía bằng BFS.
- 4 Sinh ngẫu nhiên đồ thị với nhiều kích thước khác nhau và đo thời gian chạy thực nghiệm của BFS/DFS.
- 5 Viết báo cáo ngắn: khi nào BFS/DFS không phải là thuật toán phù hợp?

- Pat Morin, *Open Data Structures*, Chapter 12: Graphs.
- MIT 6.006 Introduction to Algorithms, lecture notes on graph search.
- NetworkX documentation: graph classes and traversal algorithms.
- Cormen, Leiserson, Rivest, Stein, *Introduction to Algorithms*, chapters on elementary graph algorithms.